

FDE Mindset and Problem Structuring

Systems Thinking, Causal Reasoning, and Business Translation
for Forward Deployed AI Engineers

Practical Study Guide

Module 1

Post Graduate Certificate in Forward Deployed AI Engineering (FDE)

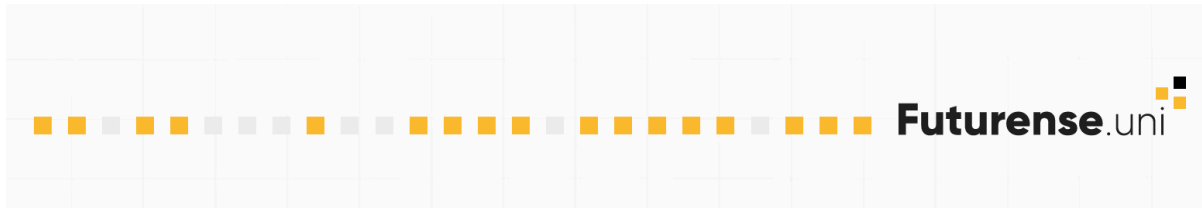
1. Introduction

A Forward Deployed AI Engineer arrives at a client site holding a model that works and leaves three months later with a system that either moved a business number or did not. Everything in between is problem structuring. This guide covers the thinking that decides which of those two outcomes happens.

Model quality stopped being the main bottleneck once strong general models became available behind an API call. The bottleneck moved to the parts around the model: what context reaches it, what it retrieves, what it remembers, what it learns from, and where the deployed system ends and the client business begins. Most failed AI deployments are not model failures. They are boundary failures, dependency failures, and translation failures.

This guide covers:

- System boundary definition, and why most reported accuracy numbers measure the wrong box
- Model-led evaluation versus system-led evaluation, and when each one lies
- The three rungs of causal reasoning: correlation, intervention, counterfactual
- Causal mapping, dependency chains, and feedback loops inside deployed AI systems
- Emergent behaviour, and how to plan for behaviour nobody specified



- Business-to-technical translation, KPI identification, and data mapping
- Technical brief composition and stakeholder alignment under disagreement

Key insight: After this guide, a student can take an ambiguous client sentence such as "we want AI to reduce costs" and produce a system boundary, a causal map, a KPI tied to a real data field, and a one-page technical brief an engineering team can build from.

2. Core Concepts

This section covers the four ideas that every later section depends on: what the role actually owns, where the system ends, what the four moving parts of a generative system are, and what "it works" means.

2.1 The FDE Mindset: Owning the Outcome, Not the Model

A research engineer owns a benchmark score. A platform engineer owns uptime. A Forward Deployed AI Engineer owns whether a specific client gets a specific business result from a specific deployment. That difference changes every decision.

Three habits separate the mindset from ordinary engineering work.

1. **Start from the decision, not the data.** The first question is never "what data do you have". It is "what decision does someone make today that this system would change". A system that produces beautiful predictions nobody acts on has zero value.
2. **Assume the client description is wrong on purpose.** Clients describe symptoms, not causes. "Our support costs too much" is a symptom. The cause might be a broken product, a confusing bill, or a policy that forces three transfers per call. An AI agent fixes none of those.
3. **Trace the whole chain before optimising any link.** A 4-point gain on the model matters only if the model is the binding constraint. Often it is not.

Watch out: Optimising the component that is easiest to measure is the most common way an FDE project burns a quarter and ships nothing. Measurability is not the same as importance.

2.2 System Boundary Definition

A system boundary is the line the engineer draws around what counts as "the thing being built". Everything inside the line is owned and can be changed. Everything outside is an assumption.

Most arguments between an AI team and a client are actually arguments about where this line sits, held by people who have never drawn it on paper.

Three boundaries exist in every deployment, and they rarely agree.

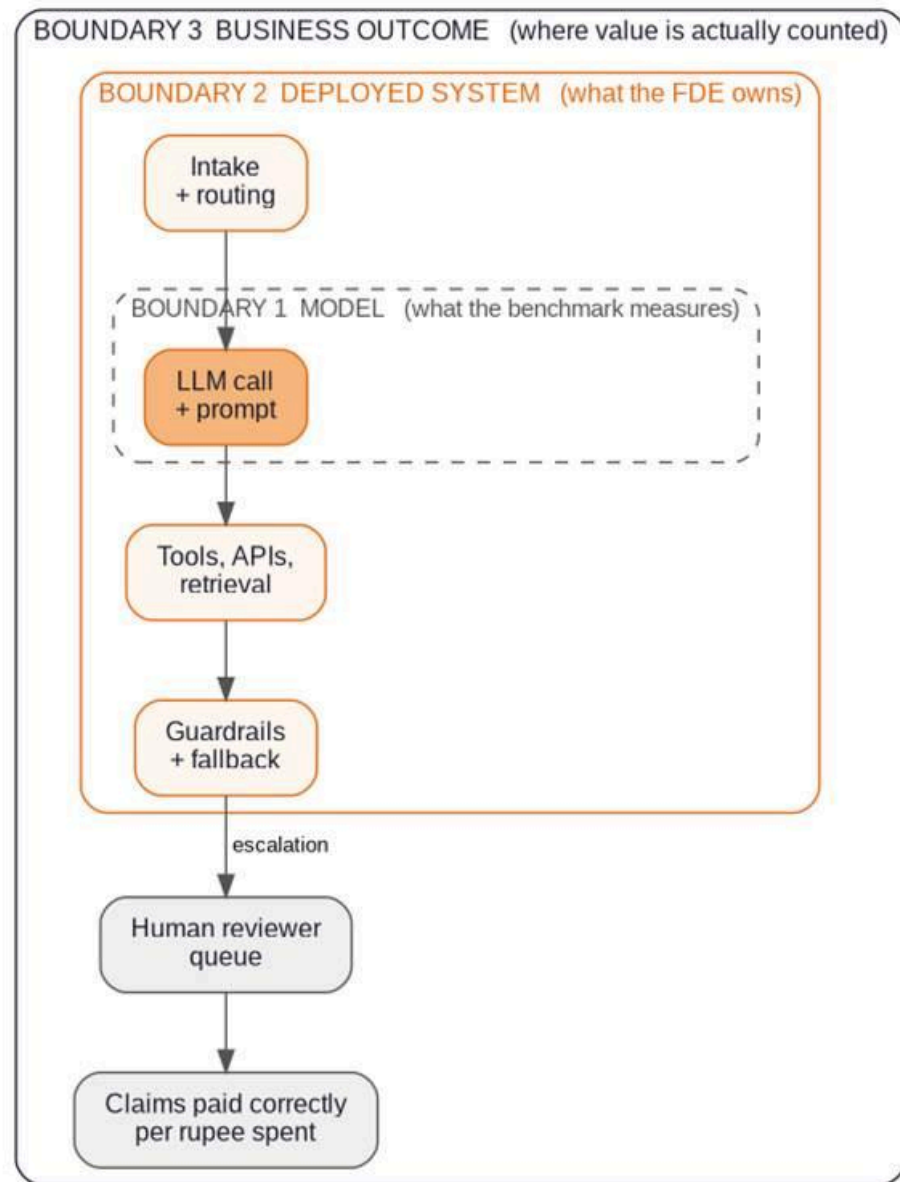
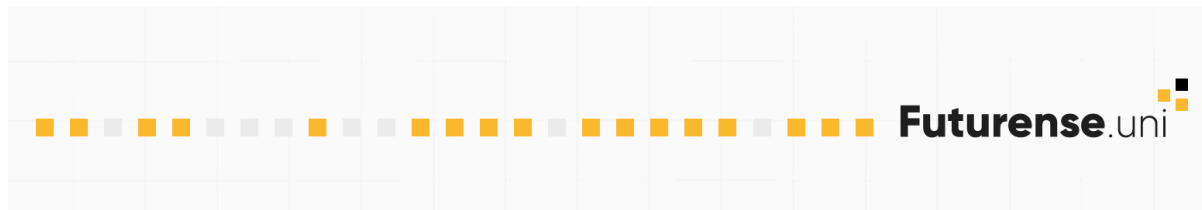


Figure 1: Three nested boundaries in a claims triage deployment. Benchmarks measure the innermost box. Clients pay for the outermost one.

The model boundary contains the prompt and the model call. Public benchmarks and vendor marketing live here. The system boundary contains everything the engineering team can



change: intake, routing, retrieval, tools, guardrails, and fallbacks. The business boundary contains the human process the system feeds into, including reviewer queues, approval steps, and the accounting rule that decides whether a rupee was saved.

Drawing the boundary is a decision, not a discovery. Draw it too small and the team ships a component nobody can use. Draw it too large and the team ends up redesigning a client business process it has no authority over. The practical rule is to draw the system boundary at the last point where the team still controls the output and the first point where the client can measure it.

Key insight: Write the boundary down as a list of "we own X" and "we assume Y" statements, then get the client to sign the assumptions. Every unwritten assumption becomes a dispute in month three.

2.3 The Generative Four and the Mental Model

Any deployed generative system, from a chatbot to a multi-step agent, is built from four moving parts. Naming them makes design arguments concrete instead of vague.

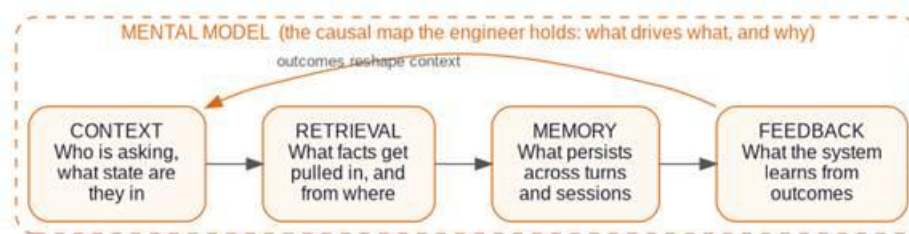
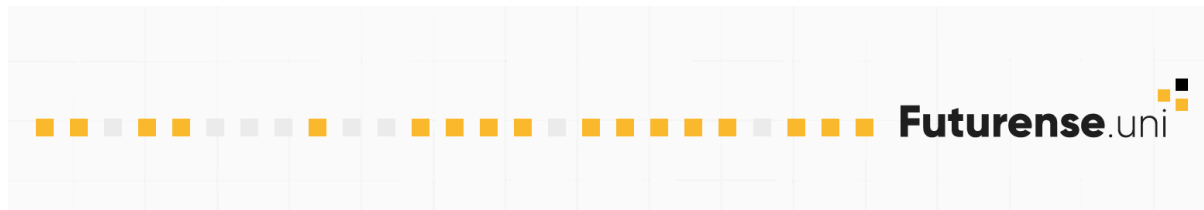


Figure 2: Context, Retrieval, Memory, and Feedback form a closed loop. The mental model is the engineer's theory of how they drive each other.

- **Context** is what the system knows about the current request: who is asking, what state they are in, what they already tried. Weak context makes every other part work harder.
- **Retrieval** is what facts get pulled in from outside the model, and from where. Retrieval decides the ceiling on factual correctness.
- **Memory** is what persists across turns and across sessions. Memory turns a stateless tool into something that feels like a colleague, and it also turns a small error into a permanent one.
- **Feedback** is what the system learns from outcomes: thumbs, corrections, escalations, downstream results. Feedback is where the loops live.

The mental model is the fifth item, and it does not sit inside the system. It is the engineer's working theory of how the four parts drive each other and drive the business number. A causal



map is the written form of that mental model. Without it, changing retrieval to fix a factual error can quietly degrade memory quality three weeks later and nobody connects the two events.

2.4 Model-Led Evaluation vs System-Led Evaluation

Model-led evaluation scores the model in isolation on a fixed dataset: accuracy, factual correctness, coherence, benchmark rank. System-led evaluation scores the deployed system on end-to-end task completion under real conditions, including retrieval misses, tool timeouts, guardrail blocks, and human handoffs.

The gap between them is not academic. A survey of evaluation practice found that around two thirds of academic studies measure the model alone, while only about one in five look at system-level interactions, and industry practice leans the other way because deployed agents fail through tool errors and cascading decisions rather than through poor sentence quality.

Dimension	Model-led evaluation	System-led evaluation
Unit measured	One model call on one input	One user task, start to finish
Typical metric	Accuracy, F1, win rate, benchmark score	Task completion rate, cost per resolved case, escalation rate
Dataset	Fixed, curated, balanced	Live traffic sample, unbalanced, messy
Catches	Reasoning errors, hallucination, format drift	Retrieval misses, tool failures, latency, bad handoffs
Misses	Everything outside the model boundary	Which component caused the failure, unless traced
Runs when	Model swap, prompt change, fine-tune	Every release, plus continuously in production

Run model-led evaluation to choose a component. Run system-led evaluation to decide whether to ship.

Watch out: A model-led score is an upper bound on system performance, never a prediction of it. Quoting a model-led number to a client as the expected result is the fastest way to lose credibility in month two.

3. Technical Deep Dive

This section covers the reasoning machinery: how to tell correlation from cause, how to map a system as a graph, how error compounds along a chain, how loops change behaviour over time, and what emergence means in practice.

3.1 The Three Rungs of Causal Reasoning

Causal reasoning is not the same thing as AI reasoning, and it is not a subset of it either. AI reasoning describes what a model does when it works through a problem in steps. Causal reasoning describes a class of questions that can only be answered with assumptions about how the world works. A model can perform causal reasoning, and a model can also reason at length and still answer a causal question wrongly, because the data alone never contains the answer.

Judea Pearl organises causal questions into three rungs. Each rung needs strictly more than the one below it.

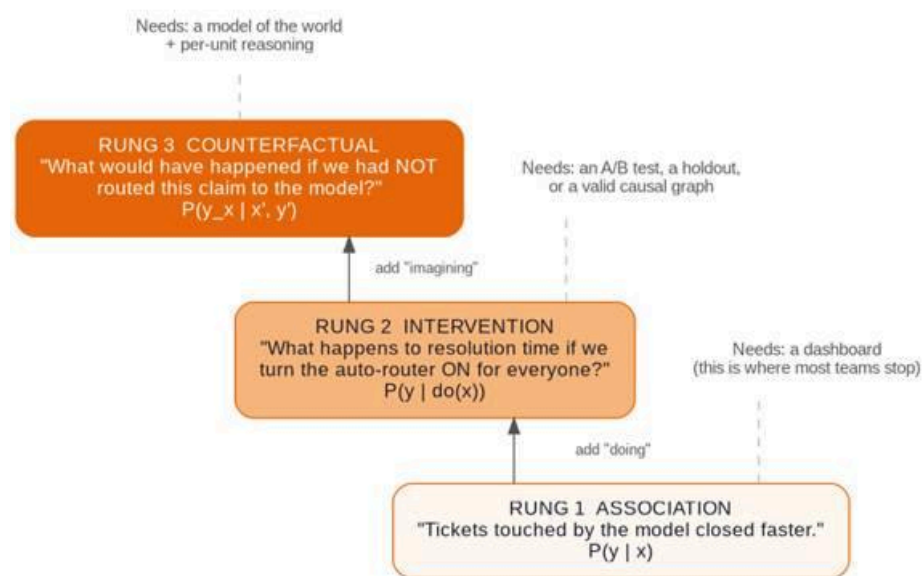
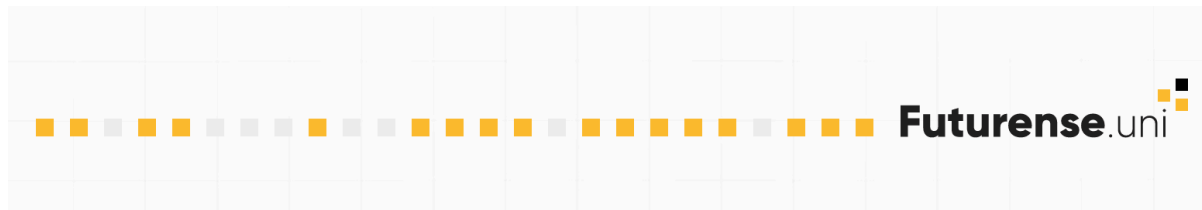


Figure 3: The three rungs of causation. Each step up requires assumptions that the data on its own cannot supply.

Rung 1: Association (seeing)

Rung 1 answers "what does the data show". It is written as a conditional probability.

$P(Y = y \mid X = x)$ read as: given that X was observed to be x , how likely is Y



Example: tickets touched by the agent closed 43% faster. This is a true statement about the log file. It says nothing about what would happen if the agent handled more tickets.

Rung 2: Intervention (doing)

Rung 2 answers "what happens if we change something". The do-operator marks the difference.

$P(Y = y \mid \text{do}(X = x))$ read as: given that X was set to x by us, how likely is Y

The distinction matters because in observational data, X was chosen by some process. If routing sends easy tickets to the agent, then observing "agent handled it" also tells you "the ticket was easy". Setting X by intervention breaks that link. An A/B test is the brute-force way to reach rung 2. Adjusting for the confounder using a valid causal graph is the analytical way.

Rung 3: Counterfactual (imagining)

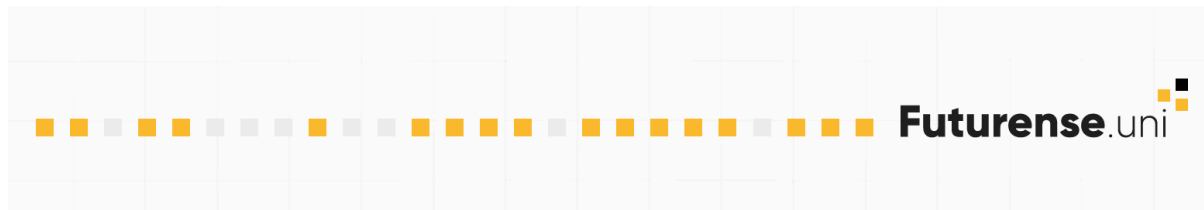
Rung 3 answers "what would have happened to this specific case, had we acted differently". It conditions on what actually happened and then asks about a world where it did not.

$P(Y_x = y \mid X = x', Y = y')$ read as: for a unit where X was x' and Y was y', what would Y have been under X = x

This is the rung clients ask about without knowing it. "Would this claim have been caught without the model" is a counterfactual. So is every attribution argument about credit for a result. Answering it needs a model of the underlying mechanism, not just a treatment effect average.

Client question	Rung needed	What it takes to answer honestly
Which customers churn most?	Rung 1	A query over historical data
Will the discount reduce churn?	Rung 2	An experiment, or a defensible causal graph
Did the discount save this account?	Rung 3	A mechanism model plus per-unit assumptions
Is the agent faster than humans?	Rung 2	Adjust for what routing sends to each arm
What is our AI ROI so far?	Rung 3	A stated counterfactual baseline, written down in advance

Match the rung to the question before promising an answer. Most ROI disputes are rung 3 questions answered with rung 1 evidence.



Key insight: The single most valuable sentence an FDE can say in a client meeting: "that is a causal question, and our current data can only answer the correlational version of it, so here is what we would need to run."

3.2 Causal Mapping

A causal map is a directed graph where nodes are things that can go up or down, and edges carry a sign. A plus sign means the two move together. A minus sign means they move in opposite directions. It is the artefact that turns a two-hour discovery conversation into something an engineer can argue with.

Building one takes four passes.

1. **List the nodes as quantities, not events.** "Escalation rate" is a node. "The outage last March" is not. Nodes must be things that can take a value.
2. **Draw only edges someone can defend.** Every edge is a claim. If nobody in the room can explain the mechanism behind an arrow, delete it or mark it as an open question.
3. **Sign every edge.** Unsigned edges hide the difference between a runaway loop and a self-correcting one.
4. **Find the cycles.** Cycles are where the interesting behaviour lives, and where the system will surprise the client.

Watch out: A causal map with no cycles usually means the interview was too short. Real deployed systems almost always feed back into their own inputs somewhere.

3.3 Dependency Chains and Compounding Error

A dependency chain is a sequence of steps where each one needs the previous one to have succeeded. Agentic AI systems are dependency chains by construction: parse, retrieve, plan, call a tool, check, write. Reliability multiplies along the chain, so it falls fast.

$$P(\text{chain succeeds}) = p_1 \times p_2 \times p_3 \times \dots \times p_n$$

Worked example with six steps at 95% each:

$$\begin{aligned} &0.95 \times 0.95 \times 0.95 \times 0.95 \times 0.95 \times 0.95 \\ &= 0.95^6 \\ &= 0.735 \end{aligned}$$

Six "excellent" steps produce a 73.5% system.

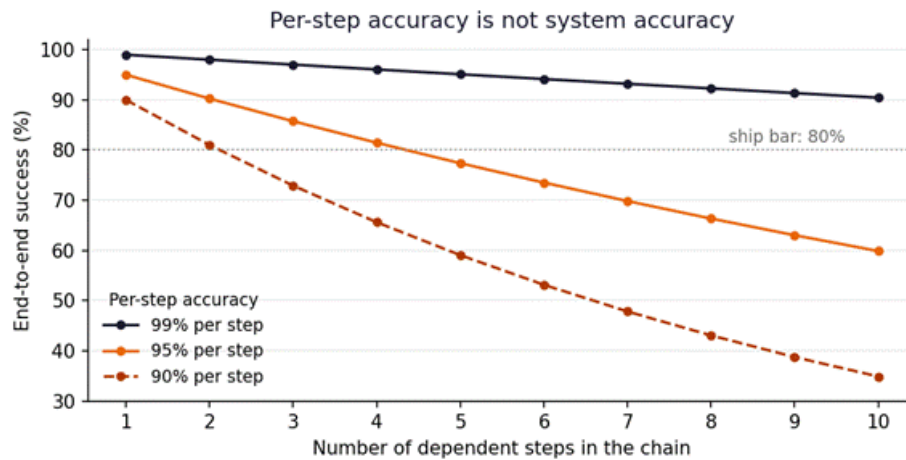
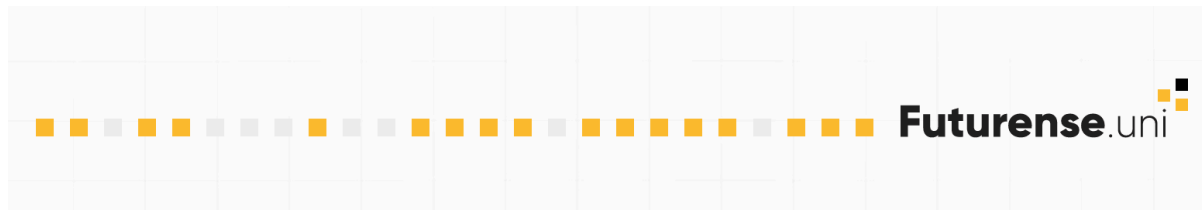


Figure 4: End-to-end success against chain length. A 95% component looks strong until six of them sit in series.

Three consequences follow directly, and all three are design decisions rather than model decisions.

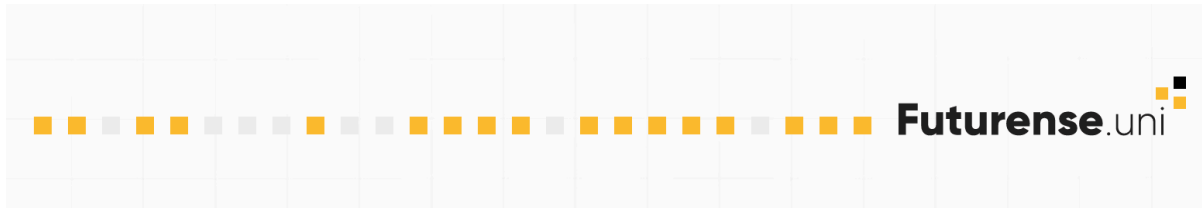
- **Shorten the chain.** Removing one step from a six-step 95% chain buys more than raising every step by one point.
- **Break the multiplication.** Retries, fallbacks, and validators convert a serial dependency into a parallel one. A step with a retry at 95% becomes roughly $1 - 0.05^2$, or 99.75%.
- **Find the binding constraint.** The lowest number in the chain dominates. Effort spent anywhere else is close to wasted.

Key insight: Ask the chain question in the first client meeting: "how many things have to go right, in order, for one unit of work to finish?" The answer sets the realistic ceiling before any model is chosen.

3.4 Feedback Loops in AI Systems

A feedback loop exists when an output of the system eventually becomes an input to the same system. Two kinds exist, and they behave in opposite ways.

A reinforcing loop (marked R) amplifies. Each pass around the loop pushes the same direction, so small changes grow. A balancing loop (marked B) dampens. Each pass pushes back toward a target, so the system self-corrects. The sign of a loop is the product of the signs of its edges: an even number of negative edges makes it reinforcing, an odd number makes it balancing.



Diagnose loops by the shape of the metric over time, not by the value on any single day.

Watch out: The most common reinforcing loop in deployed AI is the system consuming its own output: auto-generated answers get logged as resolutions, then reused as examples, then treated as ground truth. Check for this on every project.

3.5 Emergent Behaviour

Emergent behaviour is behaviour of the whole system that none of the parts specify. It shows up in two different places, and conflating them causes bad arguments.

At the model level, emergence describes capabilities that appear at larger scale and were not present at smaller scale. That claim is contested. Work from Stanford argued that many reported emergent abilities are an artefact of using discontinuous metrics such as exact match, and that switching to continuous metrics turns the sharp jump into a smooth curve. The practical lesson for an FDE is not about scaling laws at all. It is that the choice of metric can manufacture or hide a capability cliff.

At the system level, emergence is far less controversial and far more relevant. Put a language model, a retrieval index, a tool router, a cache, and a human queue together, and the combination produces behaviour that no component was designed for: agents that loop, caches that serve confidently wrong answers, reviewers who stop reading because the system is usually right.

- **Automation complacency:** reviewers approve faster over time, so the human check silently stops being a check.
- **Specification gaming:** the system finds a path that scores well on the metric without doing the task, such as closing tickets by asking a clarifying question and timing out.
- **Coordination artefacts:** two agents or two rules interact to produce a state neither anticipated, such as an infinite handoff.

Key insight: Emergent behaviour cannot be designed away, only detected early. Budget for observability and periodic behaviour audits from day one rather than treating them as a later phase.

4. Decision Frameworks

This section covers the consulting half of the role: turning business language into technical specification, choosing KPIs that can actually be measured, writing the brief, and getting stakeholders to agree.

4.1 Business-to-Technical Translation

Translation failure is the most expensive failure mode in the role, because it is invisible until delivery. Everyone nods in the kickoff, the team builds the wrong thing correctly, and the mismatch surfaces at the demo. A five-layer ladder makes the gap visible while it is still cheap to fix.

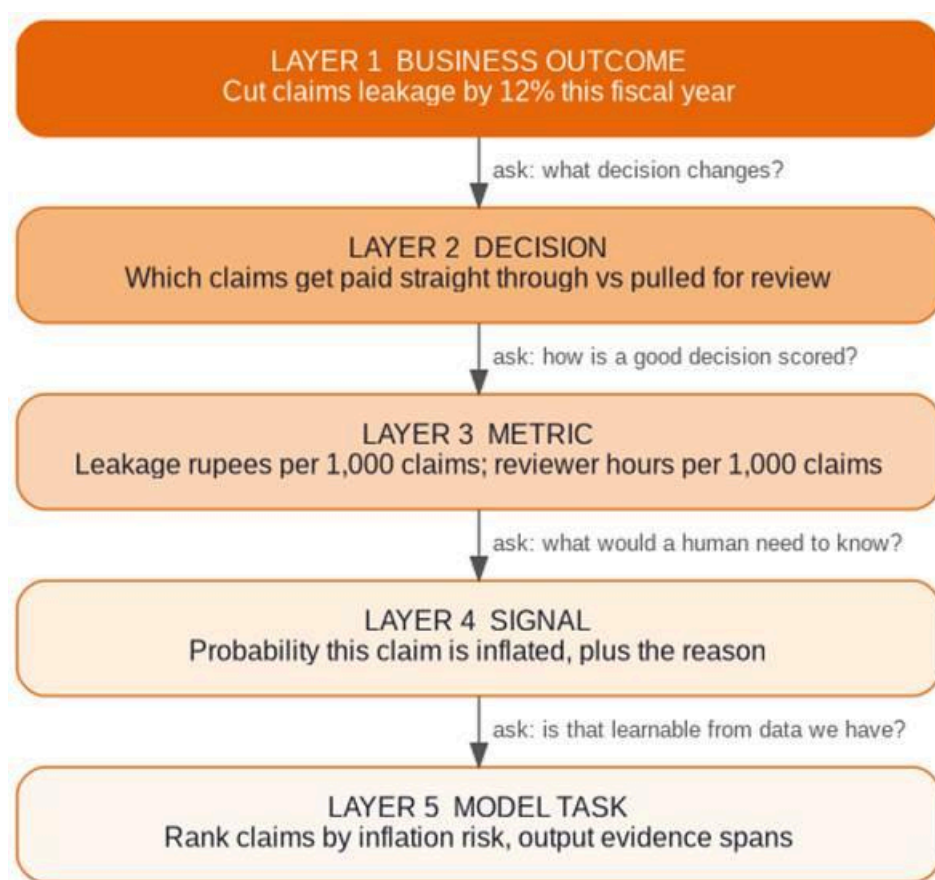
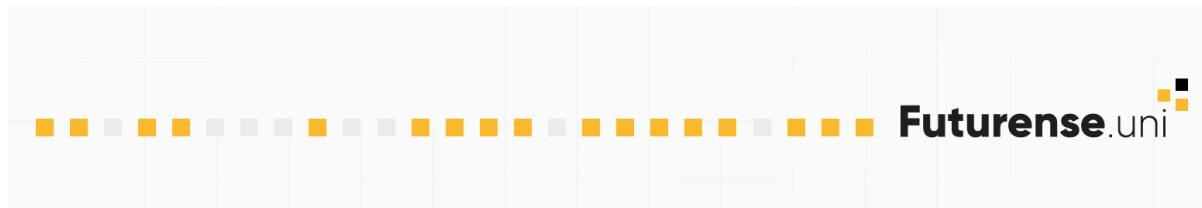


Figure 6: Each rung is one question. Skipping a rung is how teams end up building a model nobody can act on.

The discipline is that no layer may be skipped and every layer must be written in the client's own words where possible. Two checks catch most errors.

- **Downward check:** read layer 5 back to the business sponsor. If they cannot see how it produces layer 1, the chain is broken.



- **Upward check:** ask what changes at layer 2 if the layer 4 signal is perfect. If the honest answer is "nothing, we would still review everything", the project has no value path and should be restructured now.

Watch out: A signal that nobody is allowed to act on has zero business value regardless of its accuracy. Regulated industries are full of these. Check the authority to act before checking the data.

4.2 KPI Identification and Data Mapping

A KPI is only real if three things are true: it moves when the system works, it can be computed from fields that exist today, and someone owns it. Most proposed KPIs fail the second test and nobody notices until the reporting sprint.

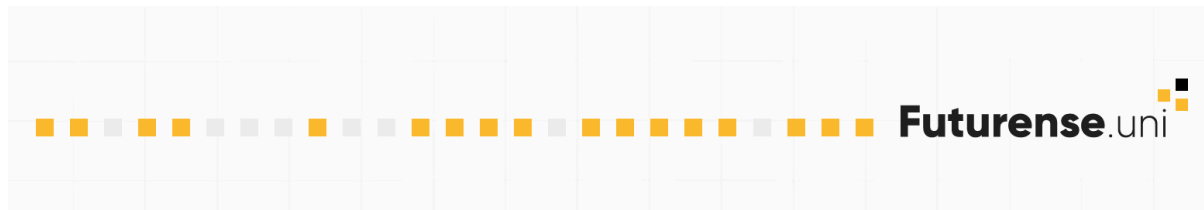
The mapping worksheet forces the check early.

Proposed KPI	Fields required	Coverage today	Verdict
Leakage per 1,000 claims	claim_amount, approved_amount, audit_flag	99%, 97%, 12%	At risk: audit_flag too sparse
Reviewer hours saved	review_start_ts, review_end_ts	88%, 61%	At risk: end timestamps missing
Straight-through rate	decision_path, claim_id	94%, 100%	Measurable today
Customer trust score	nps_response, claim_id	field absent	Blocked: needs new collection

Run this table before agreeing to any KPI. A blocked KPI is a project dependency, not a reporting detail.

Three rules keep KPI selection honest.

1. **One primary KPI, at most three guardrail metrics.** Guardrails catch the case where the primary improves by damaging something else, such as deflection rising while satisfaction falls.
2. **Define the counterfactual baseline in writing before launch.** "Compared to what" is the entire ROI argument. Agreeing to it after results exist is negotiation, not measurement.
3. **Prefer rates over counts.** Counts move with volume and reward doing more work rather than better work.



4.3 Technical Brief Composition

A technical brief is the contract between the client's intent and the engineering team's build. It is one page. Anything longer stops being read, and a brief nobody reads is worse than no brief because it creates false agreement.

Eight fields, in this order:

1. **Business outcome:** the layer 1 sentence with a number and a date.
2. **Decision changed:** who does what differently once this ships.
3. **System boundary:** what the team owns, stated as a list.
4. **Assumptions:** what the team does not own but depends on. This is the field that prevents most disputes.
5. **Primary KPI and guardrails:** with the data fields named.
6. **Counterfactual baseline:** what the comparison is, agreed in advance.
7. **Failure modes accepted:** what the system will get wrong, and what happens when it does.
8. **Out of scope:** an explicit list, because silence reads as agreement.

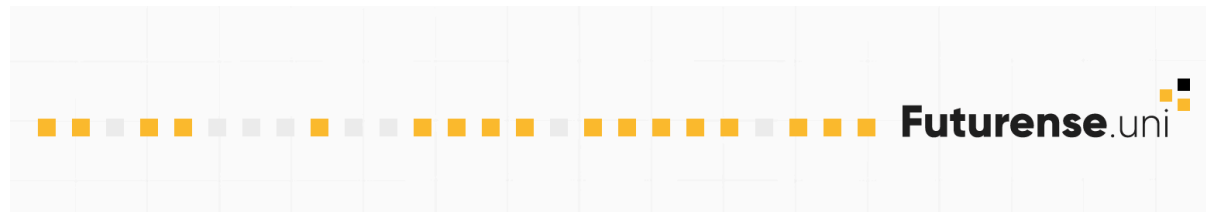
Key insight: The assumptions field and the out-of-scope field do more work than the other six combined. Write them first, and read them aloud in the room.

4.4 Stakeholder Alignment

Alignment is not agreement. Alignment means every stakeholder knows what will happen, what it costs them, and what they are expected to do, even where they would have chosen differently. Chasing full agreement stalls projects for months.

The starting move is mapping who holds what.

Role	What they control	What they fear	What earns their support
Economic sponsor	Budget and continuation	Spending with nothing to show	A dated number and a baseline
Process owner	Daily workflow and staff	Disruption and headcount questions	Being consulted on the handoff design
Data or platform owner	Access to systems	Security exposure and extra load	Narrow, specific, time-boxed access asks



End user	Actual adoption	Being blamed for machine errors	A visible override and no penalty for using it
Risk or compliance	Veto	Unexplainable decisions	Being brought in during design, not at sign-off

Map these five before the kickoff. The veto holder is the one most often discovered late.

Four techniques carry most of the weight in practice.

- **Disagree and commit, in writing.** Record the objection, record the decision, record who owns the outcome. The objection is then a logged risk instead of a recurring meeting.
- **Make the trade-off explicit, not the recommendation.** Present two options with honest costs rather than one option with a case for it. Stakeholders resist being sold and engage with being asked.
- **Pre-mortem the launch.** Ask the room to assume it failed six months out and explain why. Objections surface as analysis rather than as opposition.
- **Ship a narrow slice early.** A working system on 5% of traffic settles arguments that no slide deck resolves.

Watch out: When the same objection returns in three consecutive meetings, the issue is not information. Someone with an unaddressed stake has not been named in the room.

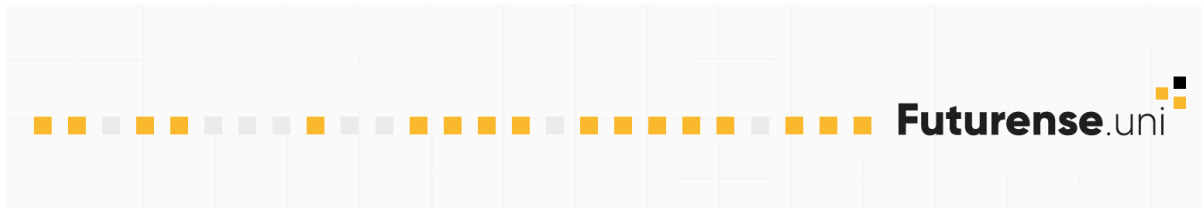
5. Code Walkthroughs

This section turns each concept into something runnable. Every block below has been executed and shows real output. These are the five scripts worth carrying into a client engagement.

5.1 Chain Reliability: Finding the Binding Constraint

Before choosing a model, work out what the pipeline can achieve at best. This script multiplies measured per-step pass rates and identifies the weakest link. Run it in the discovery phase, not after the build.

```
from math import prod
```



Each step in a claims-triage pipeline, with its measured pass rate

```
pipeline = {
    "OCR extract": 0.97, "Field parse": 0.96,
    "Policy lookup": 0.99, "LLM risk score": 0.93,
    "Rule check": 0.995, "Payout write": 0.98,
}

end_to_end = prod(pipeline.values())
weakest = min(pipeline, key=pipeline.get)
fixed = dict(pipeline, **{weakest: 0.99})

print(f"End-to-end success : {end_to_end:.3f}")
print(f"Weakest link : {weakest} ({pipeline[weakest]:.2f})")
print(f"After fixing weakest: {prod(fixed.values()):.3f}")
```

Output:

```
End-to-end success : 0.836
Weakest link       : LLM risk score (0.93)
After fixing weakest: 0.890
```

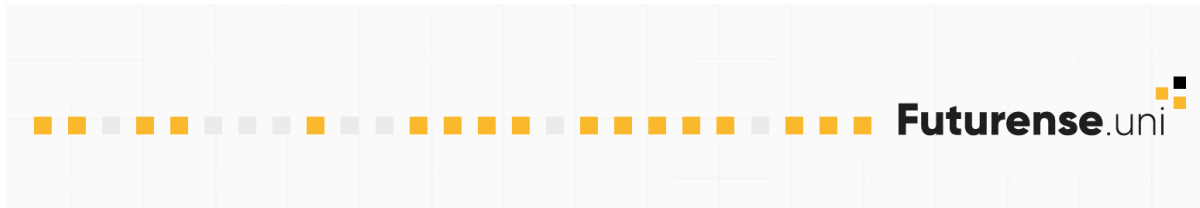
What this shows: six components that all look healthy in isolation produce an 83.6% system. Lifting the worst component from 0.93 to 0.99 buys 5.4 points end to end, which is a far better return than tuning any of the others.

5.2 Causal Map: Detecting Reinforcing and Balancing Loops

A causal map becomes useful the moment it is machine-readable. This script encodes signed edges, finds every cycle, and classifies each one by the product of its edge signs.

```
import networkx as nx

# Edge (A, B, sign): A changes B. +1 same direction, -1 opposite.
edges = [
    ("auto_answer_rate", "tickets_to_humans", -1),
    ("tickets_to_humans", "new_resolution_notes", +1),
    ("new_resolution_notes", "kb_freshness", +1),
    ("kb_freshness", "retrieval_quality", +1),
    ("retrieval_quality", "auto_answer_rate", +1),
    ("auto_answer_rate", "self_logged_wins", +1),
    ("self_logged_wins", "agent_confidence", +1),
    ("agent_confidence", "auto_answer_rate", +1),
```



```
]

graph = nx.DiGraph()
for source, target, sign in edges:
    graph.add_edge(source, target, sign=sign)

for cycle in nx.simple_cycles(graph):
    product = 1
    for i in range(len(cycle)):
        product *= graph[cycle[i]][cycle[(i + 1) % len(cycle)]]["sign"]
    kind = "REINFORCING" if product > 0 else "BALANCING"
    print(f'{{kind}} ({{len(cycle)}} hops): {\' -> \'.join(cycle)}')
```

Output:

```
BALANCING (5 hops): auto_answer_rate -> tickets_to_humans ->
                    new_resolution_notes -> kb_freshness -> retrieval_quality
REINFORCING (3 hops): auto_answer_rate -> self_logged_wins -> agent_confidence
```

What this shows: the reinforcing loop is three hops long and fires on every deploy. The balancing loop is five hops long and runs on a knowledge-base refresh cycle measured in months. The short loop wins in the short run, which is exactly the pattern that produces a strong first quarter and a bad second one.

5.3 Model-Led vs System-Led Evaluation Divergence

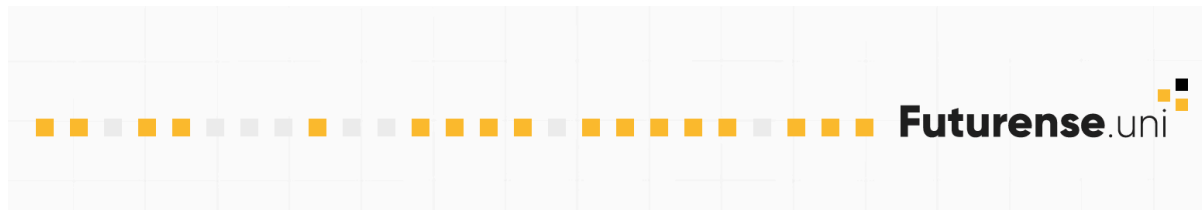
This simulation makes the evaluation gap concrete. Four stages each have a realistic pass rate. The model-led score reads only the model stage. The system-led score requires every stage to succeed, and the failure attribution shows where the real problem sits.

```
import random
random.seed(7)

STAGES = {"retrieval_hit": 0.88, "model_correct": 0.91,
          "tool_call_ok": 0.94, "guardrail_pass": 0.97}

model_wins, system_wins = 0, 0
blame = {k: 0 for k in STAGES}

for _ in range(5000):
    results = {s: random.random() < r for s, r in STAGES.items()}
    model_wins += results["model_correct"]
    if all(results.values()):
        system_wins += 1
```



```
else:
    blame[next(s for s in STAGES if not results[s])] += 1

print(f"Model-led score : {model_wins / 5000:.3f}")
print(f"System-led score: {system_wins / 5000:.3f}")
for stage, count in sorted(blame.items(), key=lambda kv: -kv[1]):
    print(f" {stage:<16} {count:>4}")
```

Output:

```
Model-led score : 0.911
System-led score: 0.732
retrieval_hit 595
model_correct      394
tool_call_ok       244
guardrail_pass     106
```

What this shows: a 91.1% model produces a 73.2% system, an 18-point gap. More importantly, retrieval causes half again as many first failures as the model does. A team that reads only the model-led score spends the next quarter swapping models and moves nothing.

Key insight: Always attribute failures to the first broken stage, not the visible symptom. The symptom is usually a bad final answer, which looks like a model problem no matter what caused it.

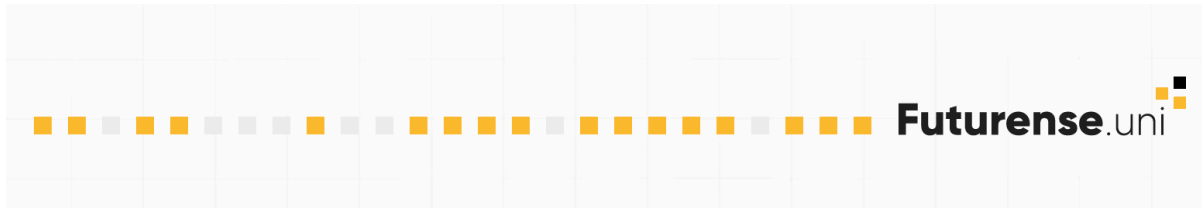
5.4 Correlation vs Intervention on Real Eval Data

This is the script that saves an FDE from making a false claim in a steering committee. Routing sends easy tickets to the agent and hard ones to humans. The raw comparison flatters the agent. Reweighting each arm to the full ticket mix reverses the conclusion.

```
# segment: (share of all tickets, agent n, agent mins, human n, human mins)
data = {
    "simple": (0.70, 560, 12, 140, 9),
    "complex": (0.30, 60, 55, 240, 40),
}

agent_n = sum(d[1] for d in data.values())
human_n = sum(d[3] for d in data.values())
naive_agent = sum(d[1] * d[2] for d in data.values()) / agent_n
naive_human = sum(d[3] * d[4] for d in data.values()) / human_n

# do(x): reweight each arm to the FULL ticket mix, not the routed mix
do_agent = sum(d[0] * d[2] for d in data.values())
```



```
do_human = sum(d[0] * d[4] for d in data.values())

print(f"Rung 1 seeing : agent {naive_agent:.1f} vs human {naive_human:.1f}")
print(f"Rung 2 doing : agent {do_agent:.1f} vs human {do_human:.1f}")
```

Output:

```
Rung 1 seeing : agent 16.2 vs human 28.6
Rung 2 doing : agent 24.9 vs human 18.3
within simple : agent 12 min vs human 9 min
within complex : agent 55 min vs human 40 min
```

What this shows: the raw numbers say the agent is 43% faster. The adjusted numbers say the agent is 36% slower. Both are computed from the same data. The agent is slower inside every single segment, and looks faster overall only because routing hands it the easy work. This reversal is Simpson's paradox, and it appears in almost every AI pilot where routing is not random.

Watch out: If a pilot used any non-random routing rule, the headline comparison is meaningless until it is adjusted. Ask "how did cases get assigned" before reading any result.

5.5 KPI to Data Field Audit

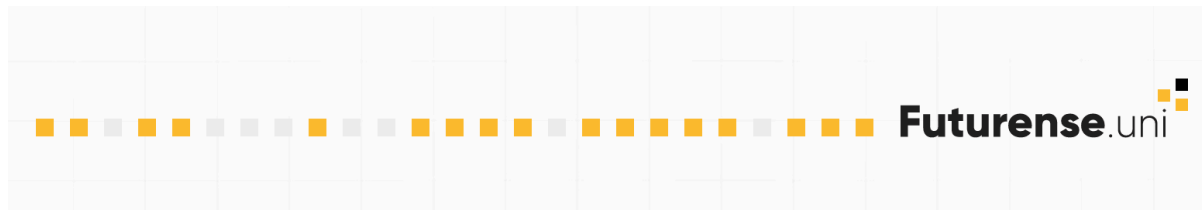
This script turns the KPI conversation into a checkable artefact. It maps each proposed KPI to the fields it needs and flags anything sparse or absent before it becomes a delivery risk.

```
kpi_needs = {
    "leakage_per_1000_claims": ["claim_amount", "approved_amount", "audit_flag"],
    "reviewer_hours_saved": ["review_start_ts", "review_end_ts"],
    "straight_through_rate": ["decision_path", "claim_id"],
    "customer_trust_score": ["nps_response", "claim_id"],
}

# Warehouse reality: field -> share of rows populated
available = {"claim_id": 1.00, "claim_amount": 0.99, "approved_amount": 0.97,
            "audit_flag": 0.12, "decision_path": 0.94,
            "review_start_ts": 0.88, "review_end_ts": 0.61}

for kpi, fields in kpi_needs.items():
    missing = [f for f in fields if f not in available]
    thin = [f for f in fields if available.get(f, 0) < 0.80]
    status = "BLOCKED" if missing else ("AT RISK" if thin else "OK")
    print(f"{status:<8} {kpi:<26} {missing or thin or 'measurable today'}")
```

Output:



AT RISK	leakage_per_1000_claims	['audit_flag']
AT RISK	reviewer_hours_saved	['review_end_ts']
OK	straight_through_rate	measurable today
BLOCKED	customer_trust_score	['nps_response']

What this shows: of four KPIs the client asked for, exactly one can be reported today. Surfacing that in week one converts three future arguments into three scoped tasks.

6. Case Studies

Four deployments, each failing through a different structural mistake. The pattern in each is worth more than the domain detail.

6.1 Case Study 1: The 91% Model That Shipped at 73%

Problem: A health insurer wants automated triage of reimbursement claims. The pilot model scores 91% on a curated set of 2,000 labelled claims. The team commits to "over 90% accuracy" in the statement of work. At go-live, operations reports that roughly one claim in four still needs full manual handling, and the sponsor calls the project a failure.

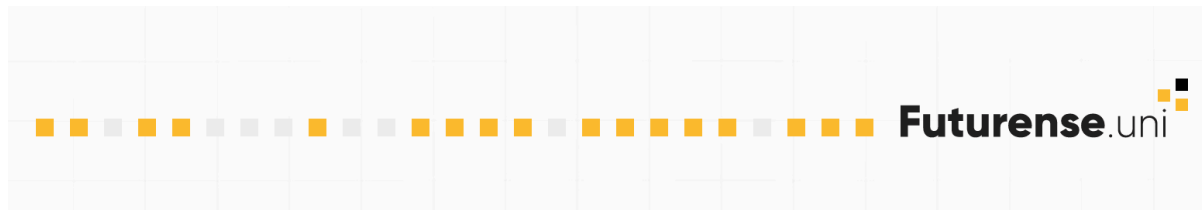
Why this needs code: the gap is arithmetic, not opinion. Multiplying the stage pass rates predicts the shipped number before launch, which turns an argument into a forecast.

Right approach: define the system boundary at intake through payout write, measure every stage on live traffic, and quote the product of the stages in the statement of work. Attribute each failure to its first broken stage. In this deployment retrieval of the correct policy version, not the model, caused the largest share of first failures because policy documents were versioned by filename and the index held two years of stale versions.

Wrong approach: quote the model-led number, then respond to the shortfall by upgrading the model. A stronger model reasoning over the wrong policy document produces a more confident wrong answer, and confident wrong answers are harder for reviewers to catch than obviously wrong ones.

Result: after fixing policy version resolution and adding one retry on the tool call, system-led completion moved from 73% to 88% with no change to the model. The contractual number was restated as an 85% straight-through rate against a named baseline.

Key insight: Write the system-led number into the contract, and write the model-led number into the appendix. The gap between them is the project.



6.2 Case Study 2: The Loop That Ate the Knowledge Base

Problem: A telecom deploys a support deflection agent. Deflection rises from 18% to 41% in the first quarter, and the team is celebrated. In the third quarter, escalation rate climbs steadily, satisfaction drops, and nobody can point to a change that caused it. No deploy correlates with the decline.

Why this needs code: the cause is structural, so it shows up in a causal map long before it shows up in a metric. The loop detection script in section 5.2 finds it in minutes from an interview transcript.

Right approach: map the loops during design. The reinforcing loop is short: auto-resolutions get logged as wins, the win rate justifies loosening the confidence threshold, more traffic auto-answers. The balancing loop is long: fewer tickets reach humans, so fewer new resolution notes are written, so the knowledge base ages, so retrieval quality falls. The fix is to break the reinforcing loop by refusing to treat auto-resolutions as ground truth, and to shorten the balancing loop by routing a fixed random 10% of deflectable tickets to humans specifically to keep generating fresh notes.

Wrong approach: treat rising escalations as a model regression and roll back the model. The model never changed. Rolling back removes the only recent evidence and delays the diagnosis by another quarter.

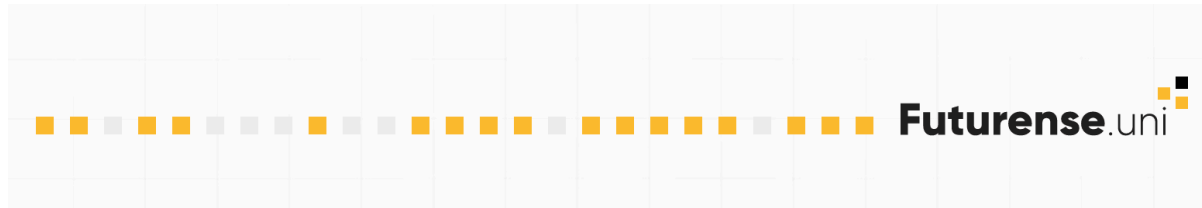
Result: holding back 10% of traffic cost about 4 points of headline deflection and stopped the decline. Escalation rate returned to its launch level within two refresh cycles. The 10% holdback also became the permanent evaluation set, which solved a second problem for free.

Watch out: Any system that consumes its own output as evidence will drift. Reserve a slice of traffic that the system never touches, and treat it as infrastructure rather than as lost value.

6.3 Case Study 3: The KPI Nobody Could Compute

Problem: A retail bank commissions an AI system to "reduce fraud losses by 15%". Eight weeks into the build, the reporting team reveals that fraud losses are booked to a quarterly consolidated ledger with a 45-day lag, are not attributable to individual transactions, and include chargebacks from three unrelated channels. The agreed KPI cannot be computed at the frequency or granularity the project needs.

Right approach: run the KPI to data field audit in week one, exactly as in section 5.5. Then walk the translation ladder to find a computable proxy that sits on the same causal path: caught-fraud value per 1,000 transactions at the decision point, measured daily, with false-positive rate as a guardrail. Agree in writing that the quarterly ledger number is the annual



outcome measure and the daily proxy is the operating measure, and state the assumed link between them.

Wrong approach: accept the headline KPI because it sounds decisive, then spend the delivery phase building a reconciliation pipeline nobody scoped. This is how a 12-week engagement becomes a 30-week one.

Result: the proxy KPI was reportable from day one of the pilot. The project delivered a measured 22% lift in caught-fraud value per 1,000 transactions with false positives held flat, and the annual ledger comparison became a confirmation rather than the only evidence.

6.4 Case Study 4: The Boundary Drawn Too Small

Problem: A manufacturer deploys visual defect detection on a production line. The model reaches 96% detection on held-out images. Scrap rate does not move. The engineering team can prove the model works and the plant manager can prove the plant is no better off. Both are correct.

Right approach: draw the business boundary and find where the value is actually counted. Detection happens at the end of the line, after the material cost is already sunk. Detecting a defect late prevents shipping it but does not prevent making it. The value path requires either moving detection upstream to the forming station or feeding detections back into machine setpoints. Both sit outside the original system boundary, which is why the pilot was scoped the way it was.

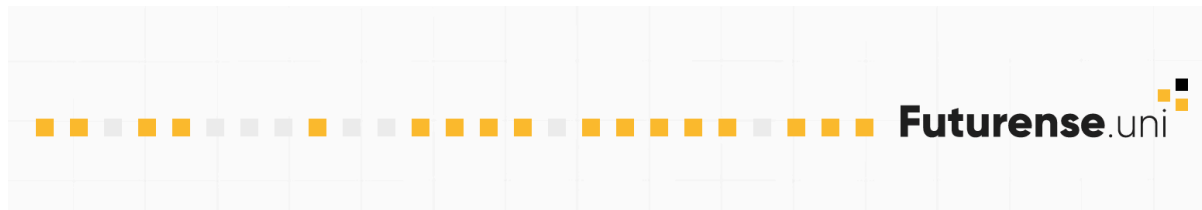
Wrong approach: raise detection to 98% and re-present. Two more points of accuracy on a measurement taken after the money is spent changes nothing, and the second demo damages credibility more than the first.

Result: rescoping to feed detections into upstream setpoint adjustment cut scrap by 11% within two months, using the same model at the same 96%. The model was never the constraint. The boundary was.

Key insight: When a strong model produces no business movement, the boundary is almost always drawn downstream of the point where value is created.

7. Common Mistakes

Eight mistakes account for most of the damage in deployment work. Each is listed with the correction.



Mistake 1: Quoting model-led accuracy as an expected business result.

Fix: Quote the product of the stage pass rates, and put the model number in an appendix.

Mistake 2: Treating a correlation from pilot logs as proof the system works.

Fix: Ask how cases were assigned. If assignment was not random, adjust before reporting anything.

Mistake 3: Drawing the system boundary around the component the team enjoys building.

Fix: Draw it at the last point the team controls output and the first point the client can measure it.

Mistake 4: Agreeing to a KPI before checking the fields exist.

Fix: Run the field coverage audit in week one and mark blocked KPIs as project dependencies.

Mistake 5: Optimising the most measurable component instead of the binding one.

Fix: Find the lowest number in the chain first. Effort elsewhere is close to wasted.

Mistake 6: Letting the system consume its own output as ground truth.

Fix: Hold back an untouched slice of traffic permanently, and never train or tune on auto-resolutions.

Mistake 7: Reading a metric on a single day instead of its shape over time.

Fix: Loops show up in the shape. A fast rise then collapse is a delayed balancing loop, not a regression.

Mistake 8: Chasing full agreement from every stakeholder before starting.

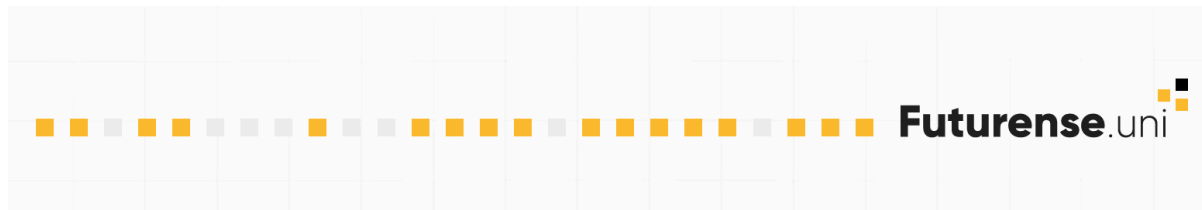
Fix: Aim for alignment: log the objection, record the decision and its owner, and ship a narrow slice.

8. Interview Preparation

These questions appear in Forward Deployed Engineer interviews in some form. The answers below are the shape of a strong response, not a script.

Q1: A client says their model is 94% accurate but the rollout is failing. Where do you start?

Answer: Start by asking what the 94% measures and what the failure looks like to a user. Almost always the 94% is a model-led score on a curated set and the failure is system-led. Map the stages between input and business outcome, get a pass rate for each, and multiply. Then attribute live failures to the first broken stage rather than the visible symptom. In most cases retrieval, tool reliability, or the human handoff dominates, and the model is not the binding constraint.



What they test: Whether the candidate separates model performance from system performance without being prompted.

Q2: How would you explain the difference between correlation and intervention to a non-technical sponsor?

Answer: Use their own data. Show that cases handled by the system resolved faster, then show that the routing rule sent easy cases to the system. Then show the within-segment comparison where the ordering reverses. Sponsors understand the routing argument immediately because they designed the routing. Finish by naming what it would take to answer the real question: a random holdout, or an adjustment using a causal graph they agree with.

What they test: Whether the candidate can teach a technical distinction using the client context rather than a textbook example.

Q3: A pilot shows a 40% improvement. The sponsor wants to scale to all traffic. What do you say?

Answer: Ask three questions before agreeing. How were pilot cases selected, because non-random selection makes the 40% unadjusted. What in the pilot was done manually that will not scale, because pilots often hide human effort. What feeds back, because a loop that is harmless at 5% of traffic can dominate at 100%. Then propose a staged ramp with a permanent untouched holdback rather than a full cutover.

What they test: Risk judgement under commercial pressure, and whether the candidate can slow a decision without blocking it.

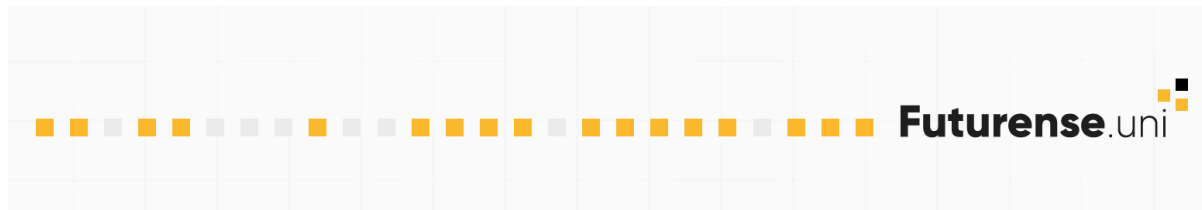
Q4: What is emergent behaviour, and does it matter for your work?

Answer: At the model level it describes capabilities appearing at scale, and the claim is contested because discontinuous metrics such as exact match can manufacture an apparent jump that continuous metrics smooth out. At the system level it is less contested and more relevant: composing a model, an index, a router, and a human queue produces behaviour nobody specified, such as reviewers approving faster over time until the human check is no longer a check. It matters because it cannot be designed away, only detected, so observability and behaviour audits belong in the initial budget.

What they test: Whether the candidate holds a nuanced view of a hyped term and can convert it into a delivery decision.

Q5: The client insists on a KPI you know cannot be measured. How do you handle it?

Answer: Do not argue about the KPI. Map it to the fields it requires and show the coverage numbers. The conversation shifts from opinion to a gap in a table. Then offer a computable proxy on the same causal path, state the assumed link between proxy and outcome in writing, and keep the original KPI as the annual confirmation measure. The client keeps their goal and the project gets something reportable from week one.



What they test: Whether the candidate can hold a line without creating a conflict, and whether they think in artefacts.

Q6: Two senior stakeholders disagree on scope and both have escalated. What do you do?

Answer: Separate the disagreement into facts, forecasts, and preferences. Facts get resolved with data in the room. Forecasts get converted into a cheap test, usually a narrow slice on real traffic. Preferences need a named decision owner, and the correct move is to make the disagreement explicit, log it, and let the owner decide. Then commit publicly, including the person who lost the argument. Repeated escalation of the same point usually means a stakeholder with a real stake has not been named yet.

What they test: Whether the candidate has operated in a real client environment or only in a friendly one.

8.1 Code Challenge

Question: Given per-stage pass rates for an agent pipeline and a budget of one engineering improvement, write a function that returns which stage to fix and the end-to-end gain from fixing it to a target rate.

```
from math import prod

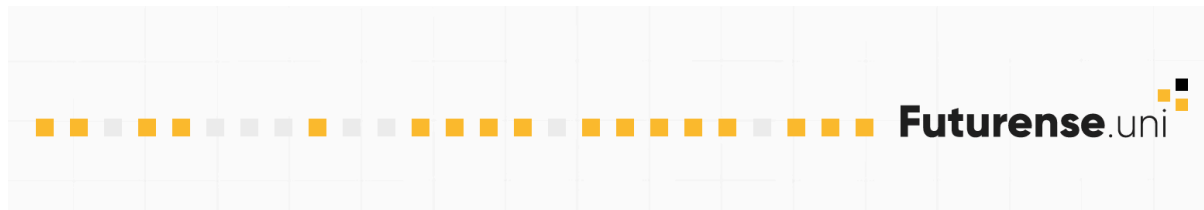
def best_fix(stages: dict, target: float = 0.99):
    """Return (stage_to_fix, gain) for the single highest-value fix."""
    base = prod(stages.values())
    options = []
    for name, rate in stages.items():
        if rate >= target:
            continue
        improved = dict(stages, **{name: target})
        options.append((name, prod(improved.values()) - base))
    return max(options, key=lambda kv: kv[1]) if options else (None, 0.0)

print(best_fix({"retrieve": 0.88, "model": 0.91,
               "tool": 0.94, "guard": 0.97}))
```

Output:

```
('retrieve', 0.09127118000000001)
```

What they look for: that the candidate compares the gain in end-to-end terms rather than picking the lowest rate by inspection. The two usually agree, but not always, and stating why they can differ is the strong answer.



9. Practice Problems

Ten questions and three assignments. The questions test recall and reasoning. The assignments test whether the frameworks survive contact with a messy brief.

9.1 Multiple Choice Questions

Q1. A pipeline has five dependent steps, each at 96% pass rate. What is the end-to-end success rate?

- A) 96%, because the steps are all equally reliable
- B) About 82%, because reliability multiplies along a chain
- C) About 80%, because the rates add and subtract from 100
- D) 92%, because only two steps can fail at once

Answer: B. B is right because dependent steps multiply: 0.96 to the power of 5 equals 0.8154 , or about 82%. A is wrong because it confuses per-step reliability with system reliability. C is wrong because failure rates do not add linearly. D is wrong because any single failure breaks the chain, not just two.

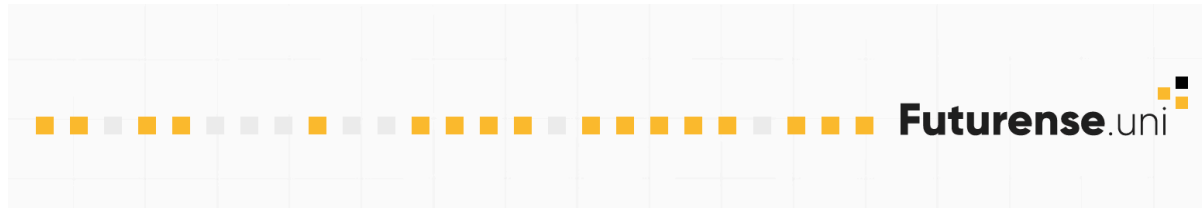
Q2. Which question sits on rung 2 of the ladder of causation?

- A) Which regions have the highest churn?
- B) Would this specific customer have stayed if we had called them?
- C) What happens to churn if we give every customer a discount?
- D) How correlated are support tickets and churn?

Answer: C. C is right because setting a treatment for everyone is an intervention, written $P(y \text{ given } \text{do}(x))$. A and D are rung 1, since both describe patterns already in the data. B is rung 3, because it asks about a specific unit in a world that did not happen.

Q3. A loop contains three edges with signs plus, minus, and minus. What kind of loop is it?

- A) Balancing, because it contains negative edges
- B) Reinforcing, because the product of the signs is positive
- C) Neither, because loops need an even number of edges
- D) Balancing, because there are more negatives than positives



Answer: B. B is right because the product of plus, minus, minus is positive, which makes the loop reinforcing. A and D are wrong because the count of negative edges does not decide the type, the parity does. C is wrong because loop length is irrelevant to loop sign.

Q4. A deployed system shows a metric that improves quickly for one quarter and then declines steadily. What should be checked first?

- A) Whether the model was silently updated by the vendor
- B) Whether a balancing loop with a long delay is correcting
- C) Whether the evaluation dataset was corrupted
- D) Whether traffic volume increased

Answer: B. B is right because rise-then-collapse is the signature of a delayed balancing loop. A is possible but would usually produce a step change on a known date, not a steady drift. C would affect measurement, not production behaviour. D changes counts but rarely produces this shape in a rate.

Q5. Which of these is a system-led evaluation metric?

- A) F1 score on a labelled intent dataset
- B) Cost per successfully resolved case in production
- C) Win rate against another model on a preference benchmark
- D) Perplexity on a held-out corpus

Answer: B. B is right because it measures a completed user task including all components and costs. A, C, and D all score the model alone on fixed data and sit inside the model boundary.

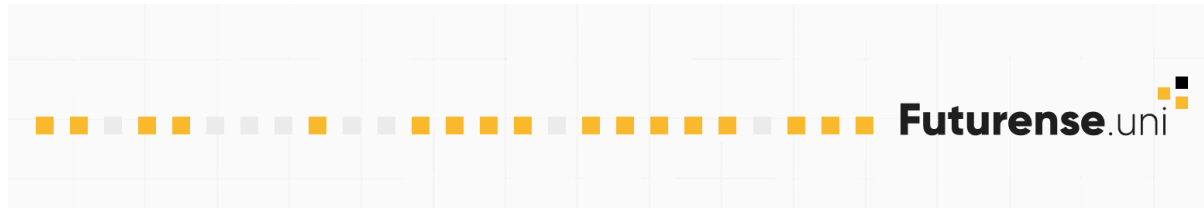
Q6. A pilot routed simple cases to the AI and complex cases to humans. The AI shows faster resolution overall. What is the correct conclusion?

- A) The AI is faster and should be scaled to all traffic
- B) The comparison is confounded by routing and must be adjusted
- C) The AI is slower and the result is a measurement error
- D) The result is valid because the sample size was large

Answer: B. B is right because non-random assignment makes the raw comparison a rung 1 statement, not a rung 2 one. A skips the adjustment entirely. C asserts a direction the raw data does not establish. D is wrong because sample size does not fix confounding, it only tightens a biased estimate.

Q7. Where should the system boundary usually be drawn?

- A) Around the model call, so the team owns only what it can control precisely
- B) At the last point the team controls output and the first point the client can measure it
- C) Around the entire client business process, so nothing is left out
- D) Wherever the client's existing dashboards already report



Answer: B. B is right because it keeps the boundary both owned and measurable. A ships a component nobody can use. C claims authority the team does not have. D lets legacy reporting decide the design, which is how the wrong number gets optimised.

Q8. What is the strongest reason to keep a permanent untouched traffic holdback?

- A) It reduces infrastructure cost by lowering model calls
- B) It preserves a clean baseline and blocks the system from consuming its own output
- C) It is required for GDPR and similar regulations
- D) It improves model accuracy through regularisation

Answer: B. B is right on both counts: a holdback gives a counterfactual baseline and prevents self-generated outputs from becoming ground truth. A is a side effect and a small one. C is not generally true. D confuses an evaluation practice with a training technique.

Q9. Reading the loop detection script in section 5.2, what does the expression `graph[cycle[i]][cycle[(i + 1) % len(cycle)]]["sign"]` retrieve?

- A) The number of edges leaving the current node
- B) The sign of the edge from the current node to the next node, wrapping to the start at the end
- C) The total sign of the entire cycle
- D) The name of the next node in the cycle

Answer: B. B is right: the modulo makes the last node connect back to the first, and the dictionary lookup returns that edge's sign attribute. A would need `graph.out_degree`. C is the running product built by the loop, not this expression. D would be the index expression alone without the sign lookup.

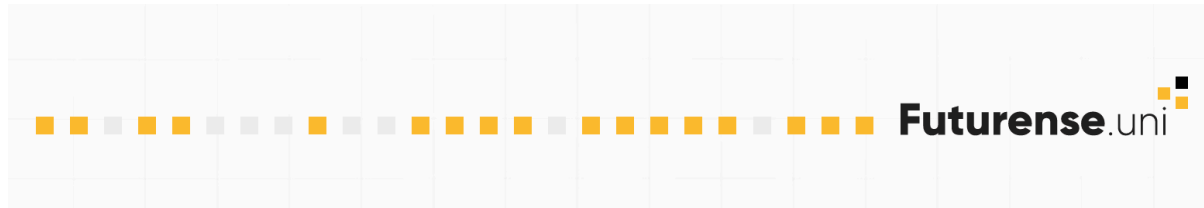
Q10. In the divergence script in section 5.3, why is failure attributed with `next(s for s in STAGES if not results[s])` rather than by listing all failed stages?

- A) It runs faster on large simulations
- B) It attributes each failure to the first broken stage in pipeline order, since later stages never ran
- C) It avoids a `KeyError` when a stage is missing
- D) It randomly samples one failure to avoid double counting

Answer: B. B is right because in a real dependency chain, once a stage fails the later stages never execute, so counting them as failures would inflate their blame. A is true but irrelevant. C describes a different problem. D is wrong because `next` takes the first match, not a random one.

9.2 Assignments

Assignment 1: Structure an Ambiguous Brief



Goal: Tests business-to-technical translation, boundary definition, and KPI mapping under realistic ambiguity.

Scenario: A logistics company says: "Our dispatch team is overwhelmed and we want AI to help them make better decisions. We think we could save two crore a year." No further detail is offered. The first workshop is in four days.

Requirements:

1. Walk the five-layer translation ladder from business outcome to model task, writing all five layers explicitly. Where a layer cannot be filled, write the exact question that would fill it.
2. Draw the three boundaries and produce a list of at least five "we own" statements and five "we assume" statements.
3. Propose one primary KPI and two guardrail metrics, and for each name the data fields required and what would make it unmeasurable.
4. List the five stakeholder roles and, for each, one sentence on what would earn their support.

Submit: A one-page technical brief using the eight fields from section 4.3, plus a one-page appendix holding the ladder, the boundary lists, and the KPI table. Two pages total.

Grading: A strong answer treats the two crore figure as a claim to be tested and states the counterfactual baseline explicitly. A weak answer accepts the number and jumps to a model architecture. The most common mistake is writing an out-of-scope list that contains only technical items, when the risky exclusions are almost always about process and authority.

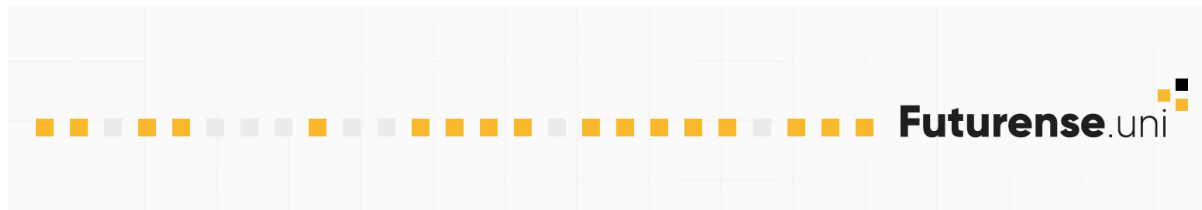
Assignment 2: Build a Loop and Chain Diagnostic

Goal: Tests whether the causal and dependency concepts can be turned into working code an FDE can carry between clients.

Scenario: A colleague is joining a new deployment next week and wants one script that takes a system description and reports its structural risks before any model is chosen.

Requirements:

1. Accept a list of pipeline stages with pass rates and a list of signed causal edges, both from a JSON or YAML file.
2. Report end-to-end success, the binding constraint, and the end-to-end gain from fixing each stage to 0.99, sorted by gain.
3. Detect every cycle, classify each as reinforcing or balancing, and report cycle length so short fast loops are visible against long slow ones.



4. Emit a plain-language risk summary of at most eight lines that a non-technical sponsor could read.

Submit: A single Python file under 120 lines, plus one example input file and the exact console output it produces.

Grading: A strong answer sorts fixes by end-to-end gain rather than by raw pass rate and explains in a comment when the two disagree. A weak answer prints the graph without interpreting it. The common mistake is treating every cycle as equally urgent when loop length and delay decide which one dominates first.

Assignment 3: Adjust a Misleading Pilot Result

Goal: Tests rung 1 versus rung 2 reasoning on data that flatters the wrong conclusion.

Scenario: A pilot report claims the AI assistant resolves cases 43% faster than human agents and recommends immediate full rollout. The routing rule sent all cases with fewer than three prior contacts to the assistant. Segment-level counts and mean handling times are available.

Requirements:

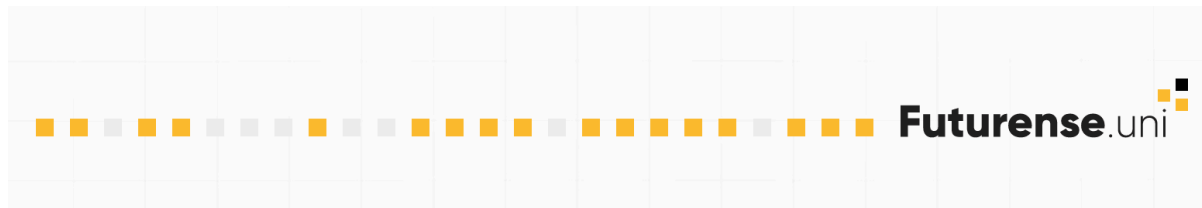
1. Compute the naive comparison and reproduce the reported figure.
2. Compute the adjusted comparison by reweighting both arms to the full case mix, and state whether the conclusion reverses.
3. Write the three sentences that would be said to the sponsor, in plain language, without using the words confounding or Simpson.
4. Specify the smallest experiment that would settle the question, including the traffic share and the run length needed.

Submit: A short script with printed output, plus a 150-word note written for a business sponsor.

Grading: A strong answer explains the reversal using the client's own routing rule as the reason and proposes a proportionate test rather than a full randomised rollout. A weak answer declares the pilot invalid and stops. The common mistake is writing the sponsor note in statistical vocabulary, which loses the room.

10. Further Learning

Reading ordered by how directly it supports the role.



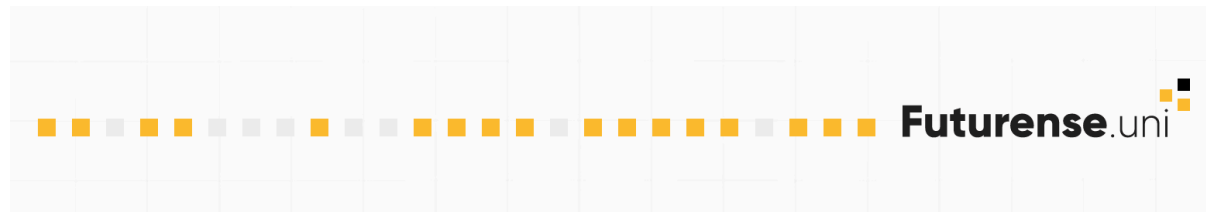
- **The Book of Why**, Judea Pearl and Dana Mackenzie. The three-rung ladder in accessible form. Read the first four chapters before the next client discovery session.
- **Thinking in Systems**, Donella Meadows. Stocks, flows, delays, and leverage points. The chapter on delays explains most of what goes wrong in quarter three of a deployment.
- **Causal Inference: The Mixtape**, Scott Cunningham. Free online. Practical treatment of adjustment, matching, and difference-in-differences with runnable code.
- **Are Emergent Abilities of Large Language Models a Mirage?** Schaeffer, Miranda, and Koyejo. Short paper on how metric choice manufactures apparent capability jumps. Useful ammunition against benchmark-driven arguments.
- **Evaluation-Driven Development and Operations of LLM Agents**. Survey covering the model-level versus system-level evaluation gap with quantified practice data.
- **Designing Data-Intensive Applications**, Martin Kleppmann. Chapters on reliability and fault tolerance give the engineering vocabulary behind dependency chain thinking.
- **DoWhy** and **CausalPy**, Python libraries. Both let a causal graph be stated explicitly and then tested against data, which is the practical form of everything in section 3.

11. Cheatsheet

One page for revision and for the flight to the client site.

Definitions

Term	One-line meaning
System boundary	The line between what the team owns and what it assumes
Model-led eval	Scores the model alone on fixed data
System-led eval	Scores the whole task end to end on live traffic
Rung 1 association	$P(y \text{ given } x)$. What the data shows
Rung 2 intervention	$P(y \text{ given } \text{do}(x))$. What happens if we act
Rung 3 counterfactual	What would have happened to this case
Causal map	Signed directed graph of what drives what



Dependency chain	Steps where each needs the previous to succeed
Reinforcing loop (R)	Even count of negative edges. Amplifies
Balancing loop (B)	Odd count of negative edges. Self-corrects
Emergent behaviour	System behaviour no component specifies
Generative Four	Context, Retrieval, Memory, Feedback

Twelve terms that cover the whole module.

Formulas

Chain success = $p_1 \times p_2 \times \dots \times p_n$

Retry on a step = $1 - (1 - p)^k$ for k attempts

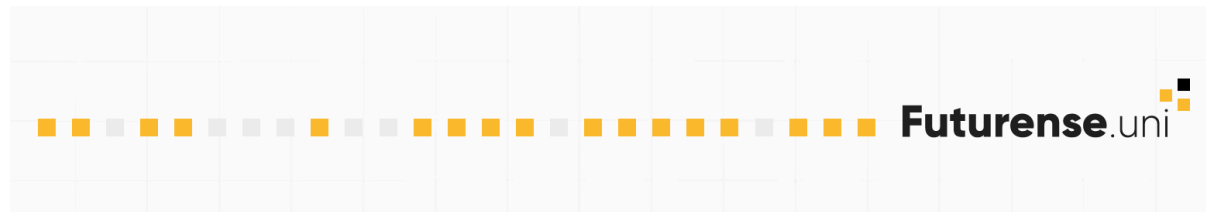
Loop sign = product of edge signs. Positive means reinforcing

Adjusted effect = sum over segments of $P(\text{segment}) \times E[Y \mid \text{arm, segment}]$

Worked example: six steps at 0.95 gives $0.95^6 = 0.735$. One retry on a 0.95 step gives $1 - 0.05^2 = 0.9975$.

Decision Table

Situation	Do this
Client quotes a model accuracy number	Ask what the stages are, multiply, quote the product
Pilot result looks strong	Ask how cases were assigned before believing it
Metric rose then fell	Look for a balancing loop with a long delay
Metric drifts with no deploy	Look for the system consuming its own output
Strong model, flat business number	The boundary is downstream of where value is made
Client proposes a KPI	Map it to fields and check coverage the same week



Same objection three meetings running	A stakeholder with a real stake is not in the room
Asked to prove ROI	Point at the counterfactual baseline agreed before launch

Scan this before any client meeting.

Eight Golden Rules

1. The model-led score is a ceiling, never a forecast.
2. Count the steps before choosing the model.
3. Every edge on a causal map is a claim someone must defend.
4. Non-random routing invalidates the headline comparison.
5. Never let the system treat its own output as ground truth.
6. Agree the counterfactual baseline in writing before launch.
7. Write the assumptions list first and read it aloud.
8. Alignment is knowing what happens next, not agreeing with it.